Solution: Binary Search

Problem Statement

Write Recursive Approach to Implement Binary Search Algorithm.

The problem is to implement the binary search algorithm using recursion. Given a sorted array and a target key, the algorithm should determine whether the key is present in the array or not.

Examples

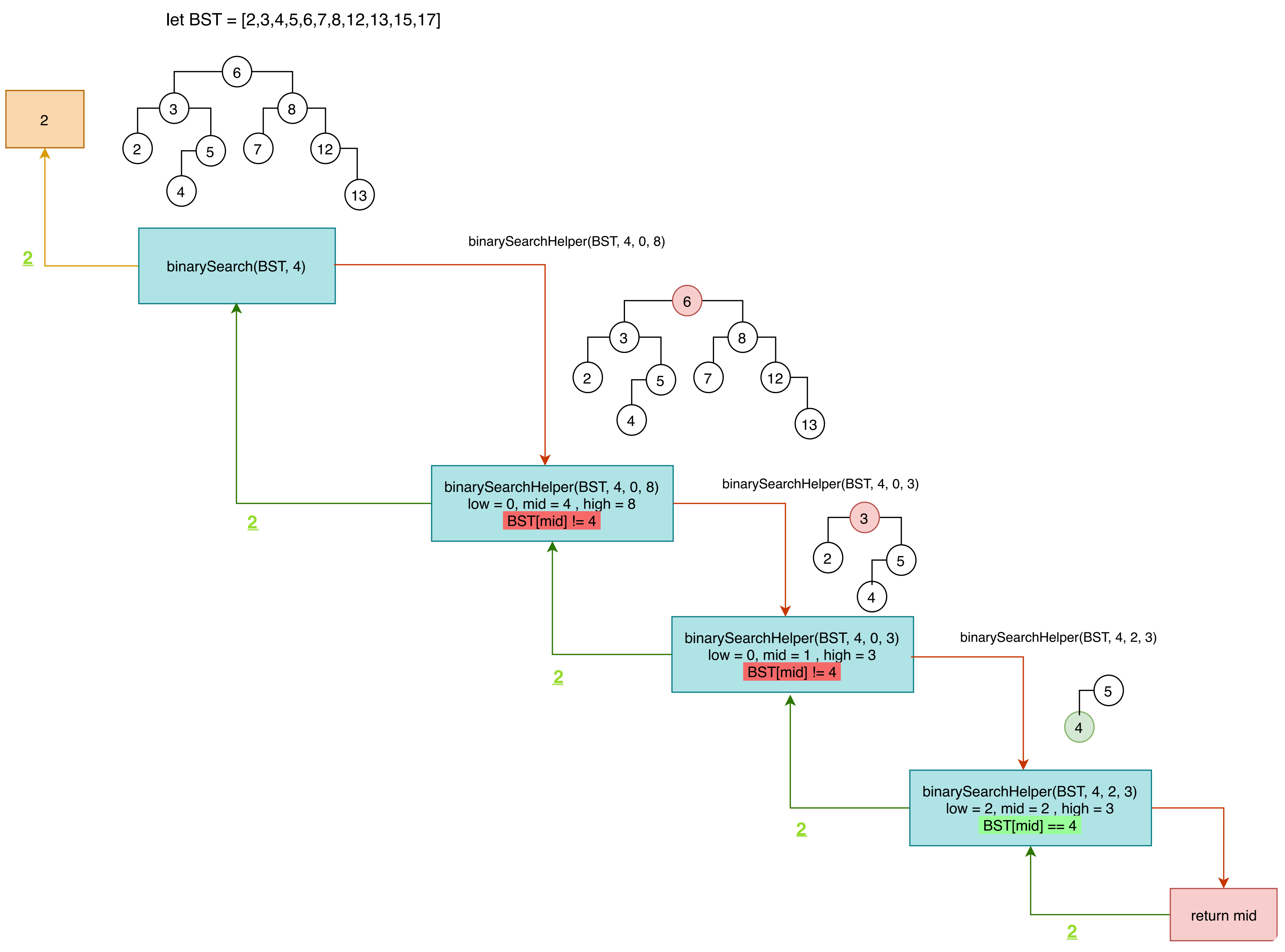
Sr #	Array	Input Key	Output
1	[1, 2, 3, 4, 5]	4	True
2	[2, 4, 6, 8, 10]	5	False
3	[3, 6, 9, 12, 15]	15	True

Solution

The algorithm follows a recursive approach to implement binary search. Here are the key points:

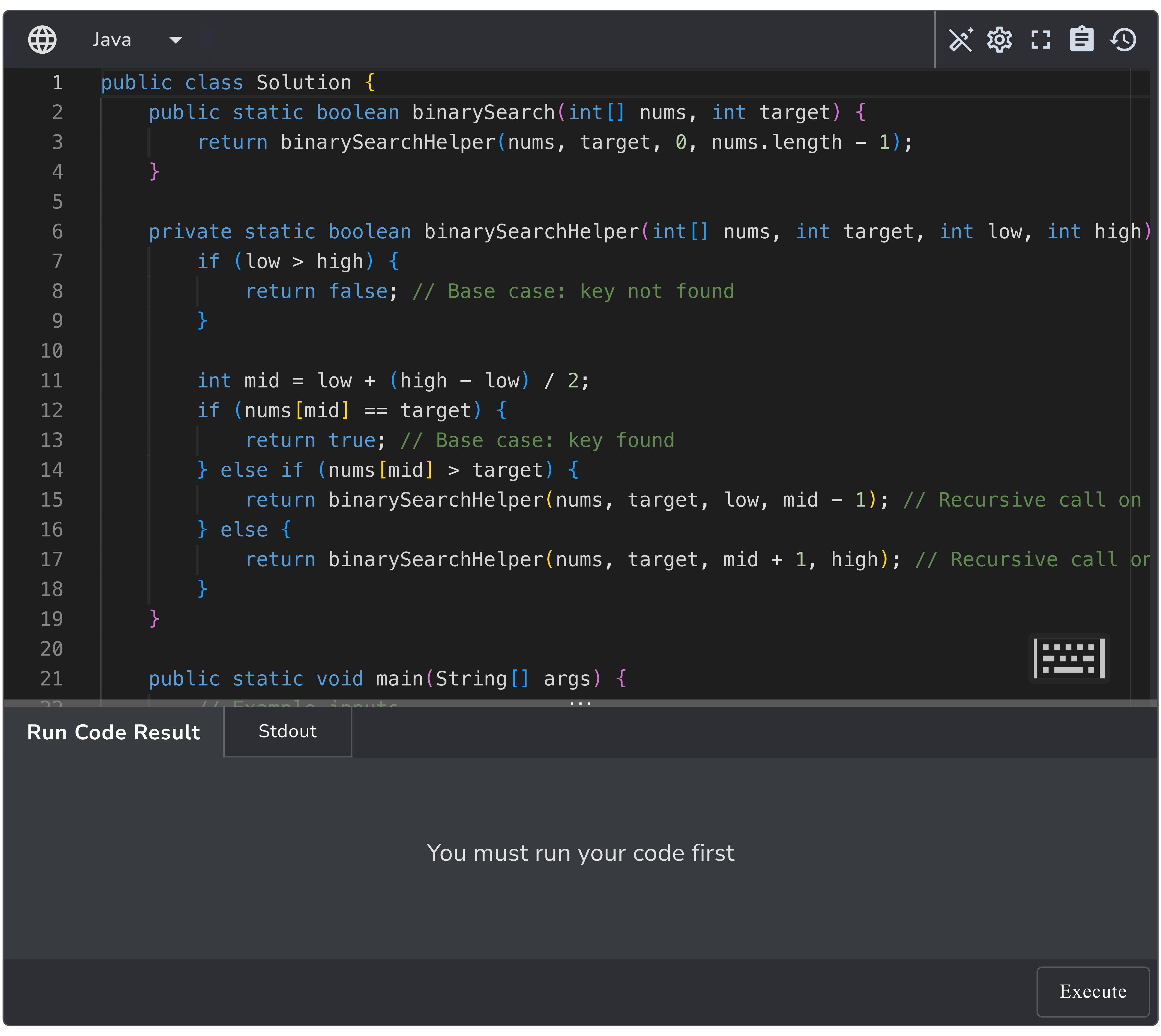
1. Base Case: If the lower bound is greater than the upper bound, return False as the key is not found.
2. Recursive Case:
   - Calculate the mid index between the lower and upper bounds.
   - Compare the element at the mid index with the target key.
     - If they are equal, return True as the key is found.
     - If the element at the mid index is greater than the target key, make a recursive call on the left half of the array with an updated upper bound of mid - 1.
     - If the element at the mid index is less than the target key, make a recursive call on the right half of the array with an updated lower bound of mid + 1.

The algorithm follows the principles of binary search, narrowing down the search space by half at each step. By utilizing recursion, it divides the array into smaller halves until the target key is found or the bounds become invalid.


let BST = [2,3,4,5,6,7,8,12,13,15,17]
2
6
3
8
2
5
7
12
4
13
2
binarySearch(BST, 4)
binarySearchHelper(BST, 4, 0, 8)
6
3
8
2
5
7
12
4
13
2
binarySearchHelper(BST, 4, 0, 8)
low = 0, mid = 4 , high = 8
BST[mid] != 4
binarySearchHelper(BST, 4, 0, 3)
3
2
5
4
2
binarySearchHelper(BST, 4, 0, 3)
low = 0, mid = 1 , high = 3
BST[mid] != 4
binarySearchHelper(BST, 4, 2, 3)
5
4
2
binarySearchHelper(BST, 4, 2, 3)
low = 2, mid = 2 , high = 3
BST[mid] == 4
return mid
2

Code

Here is the code for this algorithm:



```java
public class Solution {
    public static boolean binarySearch(int[] nums, int target) {
        return binarySearchHelper(nums, target, 0, nums.length - 1);
    }

    private static boolean binarySearchHelper(int[] nums, int target, int low, int high)
        if (low > high) {
            return false; // Base case: key not found
        }

        int mid = low + (high - low) / 2;
        if (nums[mid] == target) {
            return true; // Base case: key found
        } else if (nums[mid] > target) {
            return binarySearchHelper(nums, target, low, mid - 1); // Recursive call on
        } else {
            return binarySearchHelper(nums, target, mid + 1, high); // Recursive call or
        }
    }

    public static void main(String[] args) {
```

You must run your code first

Time and Space Complexity

The time complexity of the binary search algorithm is $O(logN)$ as the search space is halved at each recursive step. The space complexity is $O(logN)$ as well, considering the recursive function calls stored on the call stack.

← Previous
9. Binary Search

Next →
10. Inserting A New Node In A BST

☐ Mark as Completed